

실습: Hello ROS2 노드 작성

Publisher · Subscriber · Topic 통신

cchyun@gmail.com

실습 개요

- `rclpy`를 사용하여 ROS2 노드를 작성하고 토픽 통신 구조를 이해
 - 실습 목적
 - `rclpy` (Python 클라이언트 라이브러리)로 '노드' 작성
 - 노드 간 데이터를 주고받는 토픽(Topic) 통신 구조 이해
 - 핵심 도구
 - `colcon` — 빌드 도구
 - `ros2` CLI — 명령어 도구
 - VSCode — 코드 편집기
 - 최종 결과물
 - **Talker (Publisher)** — 메시지를 주기적으로 발행하는 노드
 - **Listener (Subscriber)** — 발행된 메시지를 받아 출력하는 노드

- 5단계로 Hello ROS2 노드를 완성
 - 1단계: 패키지 생성 및 구조 확인
 - 2단계: 가장 단순한 노드 작성
 - 3단계: Publisher 노드 작성 (타이머 콜백)
 - 4단계: Subscriber 노드 작성 (메시지 수신 콜백)
 - 5단계: 빌드 및 통신 확인

Part 1. 패키지 생성

1단계 — 패키지 생성 및 구조 확인

- ROS2 프로젝트의 가장 작은 단위인 패키지를 생성

- 워크스페이스 소스 폴더로 이동

```
cd ~/Workspace/ros2_ws/src
```

- Python 기반 패키지 생성

```
ros2 pkg create --build-type ament_python hello_ros2_pkg --dependencies rclpy std_msgs
```

- `--dependencies` 옵션

- 노드 작성에 필요한 라이브러리(`rclpy`, `std_msgs`)를 자동으로 `package.xml`에 추가

Part 2. 노드 기초 구조

2단계 — 가장 단순한 노드 작성

- 본격적인 통신 전, ROS2 노드의 최소 구조를 먼저 이해
 - 핵심 개념 3가지
 - `rclpy.init()` — ROS2 시스템 초기화
 - 노드를 실행하기 전 반드시 호출
 - `rclpy.Node` — 클래스 상속
 - 모든 노드는 `Node` 클래스를 상속받아 작성
 - `rclpy.spin()` — 루프 실행
 - 노드가 종료되지 않고 계속 동작하도록 유지

Part 3. Publisher 노드 작성

3단계 — Publisher 노드 작성 (타이머 콜백)

- 메시지를 주기적으로 '발행'하는 노드를 만드는 단계

- `create_publisher`

- 메시지 타입과 토픽 이름을 정의
- `(메시지타입, '토픽이름', 큐사이즈)` 형태로 호출

- `create_timer`

- 특정 주기마다 콜백 함수를 자동 실행
- `(주기_초, 콜백함수)` 형태로 호출

- 파일 위치: `hello_ros2_pkg/hello_ros2_pkg/talker.py`

Talker (Publisher) 코드 — talker.py

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String  # 기본 문자열 메시지 타입

class Talker(Node):
    def __init__(self):
        super().__init__('talker')  # 노드 이름을 'talker'로 초기화
        self.publisher_ = self.create_publisher(String, 'hello_topic', 10)
        self.timer = self.create_timer(0.5, self.timer_callback)  # 0.5초마다 실행
        self.i = 0

    def timer_callback(self):
        msg = String()
        msg.data = f'Hello ROS2! [{self.i}]'
        self.publisher_.publish(msg)  # 메시지 발행
        self.get_logger().info(f'Publishing: "{msg.data}"')  # 로그 출력
        self.i += 1

def main(args=None):
    rclpy.init(args=args)  # ROS2 초기화
    node = Talker()
    rclpy.spin(node)  # 노드 실행 유지
    node.destroy_node()  # 종료 시 노드 파괴
    rclpy.shutdown()  # ROS2 종료
```

- `setup.py`에 talker 노드를 등록해야 명령어로 실행 가능

- `console_scripts`에 진입점 추가

```
'console_scripts': [  
    'talker = hello_ros2_pkg.talker:main',  
],
```

- 등록 형식: `'실행명령어 = 패키지.모듈:함수'`

- `talker` 명령 → `hello_ros2_pkg` 패키지의 `talker.py` 파일의 `main` 함수 실행

- talker 노드를 빌드하고 실행해서 메시지 발행 확인

- 빌드 (워크스페이스 루트에서 실행)

```
colcon build --symlink-install
```

- 환경 설정 (빌드 후 반드시 실행)

```
source install/setup.bash
```

- talker 노드 실행

```
ros2 run hello_ros2_pkg talker
```

- 새 터미널을 열어 talker 노드가 실제로 발행하는지 확인
 - 활성화된 토픽 목록 확인

```
ros2 topic list
```

- 토픽 데이터를 터미널에서 직접 수신

```
ros2 topic echo /hello_topic
```

- 예상 출력

```
data: 'Hello ROS2! [0]'  
data: 'Hello ROS2! [1]'  
data: 'Hello ROS2! [2]'
```

Part 4. Subscriber 노드 작성

4단계 — Subscriber 노드 작성 (메시지 수신 콜백)

- 발행된 메시지를 '구독'하여 처리하는 노드를 만드는 단계
 - `create_subscription`
 - 구독할 토픽을 설정
 - 메시지가 도착할 때마다 실행될 콜백 함수 지정
 - (메시지타입, '토픽이름', 콜백함수, 큐사이즈) 형태로 호출
 - 핵심 포인트
 - Publisher와 동일한 토픽 이름 사용 → `'hello_topic'`
 - Publisher와 동일한 메시지 타입 사용 → `String`
 - 파일 위치: `hello_ros2_pkg/hello_ros2_pkg/listener.py`

Listener (Subscriber) 코드 — listener.py

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class Listener(Node):
    def __init__(self):
        super().__init__('listener')
        # Publisher와 동일한 토픽 이름('hello_topic')으로 구독
        self.subscription = self.create_subscription(
            String, 'hello_topic', self.listener_callback, 10)

    def listener_callback(self, msg):
        # 메시지를 받았을 때 실행되는 함수
        self.get_logger().info(f'I heard: "{msg.data}"')

def main(args=None):
    rclpy.init(args=args)
    node = Listener()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

- Part 3에서 등록한 talker에 이어 listener도 `setup.py`에 추가

- `console_scripts`에 listener 진입점 추가

```
'console_scripts': [  
    'talker = hello_ros2_pkg.talker:main',    # Part 3에서 등록 완료  
    'listener = hello_ros2_pkg.listener:main', # 지금 추가  
],
```

- 등록 형식: `'실행명령어 = 패키지.모듈:함수'`
 - `listener` 명령 → `hello_ros2_pkg` 패키지의 `listener.py` 파일의 `main` 함수 실행

- listener를 추가한 후 재빌드하고 두 터미널에서 각각 실행

- 재빌드 (워크스페이스 루트에서 실행)

```
colcon build --symlink-install
```

- 환경 설정 (빌드 후 반드시 실행)

```
source install/setup.bash
```

- 두 노드 동시 실행 (터미널 1 / 터미널 2)

```
ros2 run hello_ros2_pkg talker      # 터미널 1  
ros2 run hello_ros2_pkg listener   # 터미널 2
```

Part 5. 통신 확인

- 두 노드가 모두 실행 중일 때 시스템 상태를 확인
 - `ros2 node list` — 현재 실행 중인 모든 노드 목록

```
ros2 node list
# /talker
# /listener
```

- `ros2 topic list` — 활성화된 모든 토픽 목록

```
ros2 topic list
# /hello_topic
```

- `ros2 topic hz` — 토픽 발행 주기(Hz) 측정

```
ros2 topic hz /hello_topic
```

Part 6. 오류 해결 가이드

자주 발생하는 오류와 해결법 (1/2)

- 오류 1: `ModuleNotFoundError`

- 원인

- `setup.py` 나 `package.xml` 에 의존성(`rclpy`, `std_msgs`) 누락
- 빌드 후 `source install/setup.bash` 명령어 미실행

- 해결

- `package.xml` 의 `<depend>` 항목 확인
- 빌드 후 반드시 `source install/setup.bash` 실행

- 오류 2: 노드 실행 안 됨

- 원인: `setup.py` 에 오타, 빌드 시 에러
- 해결: 빌드 로그(`log/` 폴더) 확인

- 오류 3: 메시지 전송 안 됨

- 원인: Publisher와 Subscriber의 토픽 이름 또는 메시지 타입 불일치
- 해결: 양쪽 코드에서 토픽 이름과 메시지 타입 일치 여부 확인

```
Publisher: create_publisher(String, 'hello_topic', 10)
Subscriber: create_subscription(String, 'hello_topic', callback, 10)
```

- 오류 4: QoS Mismatch

- 원인: QoS(Quality of Service) 설정이 서로 다를 때 통신 불가
- 해결: 입문 단계에서는 큐 사이즈를 양쪽 모두 10으로 동일하게 사용

Part 7. 심화 미션

- 기본 실습을 마친 후 도전해볼 심화 과제

- 미션 1: 파라미터 활용

- `self.declare_parameter('timer_period', 0.5)` 사용
- 발행 주기를 코드 수정 없이 **외부 명령어로 변경** 가능하도록 업그레이드
- `ros2 run hello_ros2_pkg talker --ros-args -p timer_period:=1.0` 실행

- 미션 2: turtle_square 노드 작성

- `geometry_msgs/Twist` 를 `/turtle1/cmd_vel` 토픽에 발행
- 1초 전진 → 1초 제자리 회전(90°) 패턴 4회 반복 → 정사각형 경로 완성
- 힌트: `angular.z =  $\pi/2 \approx 1.571$` rad/s 로 1초 발행 → 정확히 90° 회전

ROS Node Cmake

CMake 기반 패키지 생성

- C++ 노드 작성을 위한 `ament_cmake` 빌드 타입 패키지를 생성

- 워크스페이스 소스 폴더로 이동

```
cd ~/Workspace/ros2_ws/src
```

- CMake 기반 패키지 생성

```
ros2 pkg create --build-type ament_cmake hello_cmake_pkg --dependencies rclcpp std_msgs
```

- Python 패키지와 차이점

- `ament_python` → `ament_cmake` 로 변경
- `rclpy` 대신 `rclcpp` (C++ 클라이언트 라이브러리) 사용
- `setup.py` 대신 `CMakeLists.txt` 로 빌드 구성

- `ros2 pkg create` 실행 후 생성되는 폴더/파일 구조

```
hello_cmake_pkg/  
├── CMakeLists.txt      ← 빌드 규칙 정의 (실행파일 등록)  
├── package.xml         ← 패키지 메타정보 및 의존성  
├── include/  
│   └── hello_cmake_pkg/ ← 헤더 파일 위치  
└── src/                ← C++ 소스 파일 위치
```

- 소스 파일은 `src/` 폴더에 작성
- `CMakeLists.txt`에 실행파일을 등록해야 빌드 및 실행 가능

Talker (Publisher) 코드 — talker.cpp

```
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

class Talker : public rclcpp::Node {
public:
    Talker() : Node("talker"), count_(0) {
        publisher_ = this->create_publisher<std_msgs::msg::String>("hello_topic", 10);
        timer_ = this->create_wall_timer(
            std::chrono::milliseconds(500),
            std::bind(&Talker::timer_callback, this));
    }

private:
    void timer_callback() {
        auto msg = std_msgs::msg::String();
        msg.data = "Hello ROS2! [" + std::to_string(count_++) + "]";
        publisher_->publish(msg);
        RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", msg.data.c_str());
    }
    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
    rclcpp::TimerBase::SharedPtr timer_;
    size_t count_;
};

int main(int argc, char * argv[]) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<Talker>());
    rclcpp::shutdown();
    return 0;
}
```

- `src/talker.cpp` 을 빌드 대상으로 등록하고 의존 라이브러리를 연결
 - `find_package` 아래에 다음 항목을 추가

```
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

# 실행파일 추가
add_executable(talker src/talker.cpp)
ament_target_dependencies(talker rclcpp std_msgs)

# 설치 경로 지정
install(TARGETS talker DESTINATION lib/${PROJECT_NAME})

ament_package()
```

- `add_executable` — 소스 파일을 컴파일하여 실행파일 생성
- `ament_target_dependencies` — ROS2 패키지 의존성 자동 연결
- `install(TARGETS ...)` — 빌드 결과물을 `lib/` 에 설치

Listener (Subscriber) 코드 — listener.cpp

```
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

class Listener : public rclcpp::Node {
public:
    Listener() : Node("listener") {
        subscription_ = this->create_subscription<std_msgs::msg::String>(
            "hello_topic", 10,
            std::bind(&Listener::listener_callback, this, std::placeholders::_1));
    }

private:
    void listener_callback(const std_msgs::msg::String & msg) {
        RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg.data.c_str());
    }
    rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};

int main(int argc, char * argv[]) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<Listener>());
    rclcpp::shutdown();
    return 0;
}
```


- talker에 이어 listener도 `CMakeLists.txt`에 등록

```
# talker (Part 3에서 등록 완료)
add_executable(talker src/talker.cpp)
ament_target_dependencies(talker rclcpp std_msgs)

# listener 추가
add_executable(listener src/listener.cpp)
ament_target_dependencies(listener rclcpp std_msgs)

# 두 실행파일 함께 설치
install(TARGETS talker listener
        DESTINATION lib/${PROJECT_NAME})
```

- 워크스페이스 루트에서 빌드 후 두 노드를 각각 실행

- 빌드

```
cd ~/Workspace/ros2_ws  
colcon build --packages-select hello_cmake_pkg
```

- 환경 설정

```
source install/setup.bash
```

- 두 노드 동시 실행 (터미널 1 / 터미널 2)

```
ros2 run hello_cmake_pkg talker      # 터미널 1  
ros2 run hello_cmake_pkg listener    # 터미널 2
```



- 두 방식의 핵심 차이점 정리

항목	Python (<code>ament_python</code>)	CMake (<code>ament_cmake</code>)
언어	Python	C++
빌드 설정	<code>setup.py</code>	<code>CMakeLists.txt</code>
클라이언트 라이브러리	<code>rclpy</code>	<code>rclcpp</code>
실행파일 등록	<code>console_scripts</code>	<code>add_executable</code> + <code>install</code>
빌드 속도	빠름 (스크립트)	느림 (컴파일)
실행 성능	보통	빠름